

Name : Vikas Bhagat

Div : A

Branch : Comps

Date Of Performance : 17/09/2025

Date Of Submission : 18/09/2025

Roll . no. : 08

Experiment 4: Markov Chain Modeling

Title	Markov Chain Simulation
Objective	To simulate a Markov chain and compute state probabilities over time.
Theory	Markov chains are models that describe systems which transition between states with probabilistic rules that depend only on the

	current state.
Method	Define the transition matrix and initial state. Simulate state transitions over a number of steps. Plot the state probabilities over time.
Code	<pre>import numpy as np import matplotlib.pyplot as plt import seaborn as sns import pandas as pd class MarkovChain: def __init__(self, transition_matrix, state_labels): self.P = np.array(transition_matrix) self.states = state_labels self.n = self.P.shape[0] self.validate() def validate(self): if self.P.shape[0] != self.P.shape[1]: raise ValueError("Transition matrix must be square.") for i, row in enumerate(self.P): if not np.isclose(np.sum(row), 1.0): raise ValueError(f"Row {i} does not sum to 1.") def step(self, state_vector): return np.dot(state_vector, self.P) def simulate(self, initial_state, steps): state = np.array(initial_state) states_over_time = [state] for _ in range(steps): state = self.step(state) states_over_time.append(state) return np.array(states_over_time) def compute_stationary_distribution(self): eigvals, eigvecs = np.linalg.eig(self.P.T) idx = np.argmin(np.abs(eigvals - 1)) stationary = np.real(eigvecs[:, idx]) stationary /= np.sum(stationary) return stationary</pre>

```

def simulate_random_walk(self, start_idx, steps):
    history = [start_idx]
    current = start_idx
    for _ in range(steps):
        current = np.random.choice(self.n, p=self.P[current])
        history.append(current)
    return history

def display_transition_matrix(P, labels):
    df = pd.DataFrame(P, index=labels, columns=labels)
    print("Transition Matrix (Food Choices):")
    print(df.round(3))
    print()

def display_state_table(states_over_time, labels):
    df = pd.DataFrame(states_over_time, columns=labels)
    df.index.name = "Day"
    print("Probability of Each Food Choice Over Time:")
    print(df.round(4))
    print()

def plot_probabilities(states_over_time, labels):
    plt.figure(figsize=(10, 6))
    for i in range(len(labels)):
        plt.plot(states_over_time[:, i], label=labels[i])
    plt.title("Food Choice Probabilities Over Time")
    plt.xlabel("Day")
    plt.ylabel("Probability")
    plt.legend()
    plt.grid(True)
    plt.show()

def plot_heatmap(P, labels):
    plt.figure(figsize=(6, 5))
    sns.heatmap(P, annot=True, cmap="YlOrRd",
xticklabels=labels, yticklabels=labels)
    plt.title("Transition Matrix Heatmap (Food Preferences)")
    plt.xlabel("Next Day's Choice")
    plt.ylabel("Today's Choice")
    plt.show()

def plot_random_walk(history, labels):
    time = list(range(len(history)))
    foods = [labels[i] for i in history]
    numeric = history

    plt.figure(figsize=(12, 4))
    plt.plot(time, numeric, marker='o')
    plt.yticks(range(len(labels)), labels)

```

```

plt.xlabel("Day")
plt.ylabel("Food Choice")
plt.title("Random Walk: Actual Food Eaten Over Time")
plt.grid(True)
plt.show()

print("Random Walk Food Sequence:")
print(" → ".join(foods))
print()

def main():
    # Define food labels
    food_labels = ["Pizza", "Burger", "Hotdog"]

    # Define transition matrix
    transition_matrix = [
        [0.6, 0.3, 0.1], # Pizza → Pizza, Burger, Hotdog
        [0.2, 0.7, 0.1], # Burger → ...
        [0.3, 0.3, 0.4], # Hotdog → ...
    ]

    # Initial state: starts with Pizza
    initial_state = [1, 0, 0]
    steps = 20

    # Create Markov chain
    mc = MarkovChain(transition_matrix, food_labels)

    # Display transition matrix
    display_transition_matrix(mc.P, mc.states)

    # Simulate expected probabilities
    states_over_time = mc.simulate(initial_state, steps)
    display_state_table(states_over_time, mc.states)
    plot_probabilities(states_over_time, mc.states)

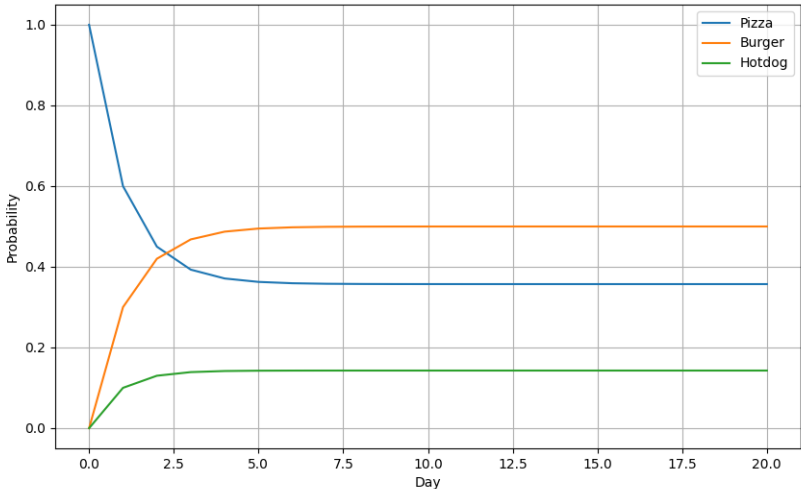
    # Heatmap
    plot_heatmap(mc.P, mc.states)

    # Stationary distribution
    stationary = mc.compute_stationary_distribution()
    print("Stationary Distribution (Long-Term Preferences):")
    for label, prob in zip(mc.states, stationary):
        print(f"{label}: {prob:.4f}")
    print()

    # Simulate random walk
    run_random_walk = True
    if run_random_walk:
        start_idx = 0 # Starting with Pizza

```

	<pre>walk_steps = 30 walk_history = mc.simulate_random_walk(start_idx, walk_steps) plot_random_walk(walk_history, mc.states) if __name__ == "__main__": main()</pre>
Output	The plot shows the probabilities of being in each state over time,demonstrating how the state probabilities converge. <pre>complab3-17@complab3-17:~/Documents/time\$ cd nsions/ms-python.debugpy-2025.10.0-linux-x64/ e/main.py Transition Matrix (Food Choices): Pizza Burger Hotdog Pizza 0.6 0.3 0.1 Burger 0.2 0.7 0.1 Hotdog 0.3 0.3 0.4 Probability of Each Food Choice Over Time: Pizza Burger Hotdog Day 0 1.0000 0.0000 0.0000 1 0.6000 0.3000 0.1000 2 0.4500 0.4200 0.1300 3 0.3930 0.4680 0.1390 4 0.3711 0.4872 0.1417 5 0.3626 0.4949 0.1425 6 0.3593 0.4980 0.1428 7 0.3580 0.4992 0.1428 8 0.3575 0.4997 0.1428 9 0.3573 0.4999 0.1429 10 0.3572 0.4999 0.1429 11 0.3572 0.5000 0.1429 12 0.3572 0.5000 0.1429 13 0.3571 0.5000 0.1429 14 0.3571 0.5000 0.1429 15 0.3571 0.5000 0.1429 16 0.3571 0.5000 0.1429 17 0.3571 0.5000 0.1429 18 0.3571 0.5000 0.1429 19 0.3571 0.5000 0.1429 20 0.3571 0.5000 0.1429</pre>

	Figure 1 Food Choice Probabilities Over Time <table><caption>Estimated data from Figure 1: Food Choice Probabilities Over Time</caption><thead><tr><th>Day</th><th>Pizza</th><th>Burger</th><th>Hotdog</th></tr></thead><tbody><tr><td>0.0</td><td>1.00</td><td>0.00</td><td>0.00</td></tr><tr><td>1.0</td><td>0.60</td><td>0.30</td><td>0.10</td></tr><tr><td>2.0</td><td>0.45</td><td>0.45</td><td>0.10</td></tr><tr><td>3.0</td><td>0.40</td><td>0.48</td><td>0.12</td></tr><tr><td>4.0</td><td>0.38</td><td>0.50</td><td>0.12</td></tr><tr><td>5.0</td><td>0.37</td><td>0.50</td><td>0.13</td></tr><tr><td>10.0</td><td>0.36</td><td>0.50</td><td>0.14</td></tr><tr><td>15.0</td><td>0.36</td><td>0.50</td><td>0.14</td></tr><tr><td>20.0</td><td>0.36</td><td>0.50</td><td>0.14</td></tr></tbody></table>	Day	Pizza	Burger	Hotdog	0.0	1.00	0.00	0.00	1.0	0.60	0.30	0.10	2.0	0.45	0.45	0.10	3.0	0.40	0.48	0.12	4.0	0.38	0.50	0.12	5.0	0.37	0.50	0.13	10.0	0.36	0.50	0.14	15.0	0.36	0.50	0.14	20.0	0.36	0.50	0.14
Day	Pizza	Burger	Hotdog																																						
0.0	1.00	0.00	0.00																																						
1.0	0.60	0.30	0.10																																						
2.0	0.45	0.45	0.10																																						
3.0	0.40	0.48	0.12																																						
4.0	0.38	0.50	0.12																																						
5.0	0.37	0.50	0.13																																						
10.0	0.36	0.50	0.14																																						
15.0	0.36	0.50	0.14																																						
20.0	0.36	0.50	0.14																																						
Quiz	Define a Markov chain. A Markov chain is a mathematical model that describes a sequence of possible events where: • The probability of each event depends only on the state attained in the previous event – not the full history. • This is called the Markov property A Markov chain consists of: • A finite or countable set of states: $S=\{s_1,s_2,...,s_n\}$ • A transition matrix P, where each element P_{ij} represents the probability of moving from state i to state j What is the stationary distribution of a Markov chain? A stationary distribution is a probability vector π such that: $\pi P=\pi$ To find π, solve: $\pi P=\pi \quad \text{and} \quad i \sum \pi_i=1$																																								
Assignm	Implement a Markov chain for a weather prediction model.																																								

en t

Compute the stationary distribution for the weather prediction model.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# Define weather states
states = ["Sunny", "Cloudy", "Rainy"]

# Transition matrix
P = np.array([
    [0.7, 0.2, 0.1], # Sunny → Sunny, Cloudy, Rainy
    [0.3, 0.4, 0.3], # Cloudy →
    [0.2, 0.5, 0.3]  # Rainy →
])

# Display transition matrix
print("Transition Matrix:")
for i, row in enumerate(P):
    print(f'{states[i]} → {dict(zip(states, row.round(2)))}')
print()

# Simulate weather over time starting from 'Sunny'
initial_state = np.array([1, 0, 0]) # 100% Sunny at start
steps = 30

state_probs = [initial_state]
current_state = initial_state.copy()

for _ in range(steps):
    next_state = np.dot(current_state, P)
    state_probs.append(next_state)
    current_state = next_state

state_probs = np.array(state_probs)

# Plotting probabilities over time
plt.figure(figsize=(10, 6))
for i in range(3):
    plt.plot(state_probs[:, i], label=states[i])
plt.title("Weather Prediction Over Time (Markov Chain)")
plt.xlabel("Day")
plt.ylabel("Probability")
plt.legend()
plt.grid(True)
plt.show()

# -----
# Compute stationary distribution
# -----
# Solve  $\pi P = \pi$ , with  $\sum(\pi) = 1$ 
```

```

# Transpose P and subtract Identity
A = np.transpose(P) - np.eye(3)

# Append constraint: sum of n = 1
A = np.vstack([A, np.ones(3)])
b = np.array([0, 0, 0, 1])

# Solve the linear system
stationary_dist = np.linalg.lstsq(A, b, rcond=None)[0]

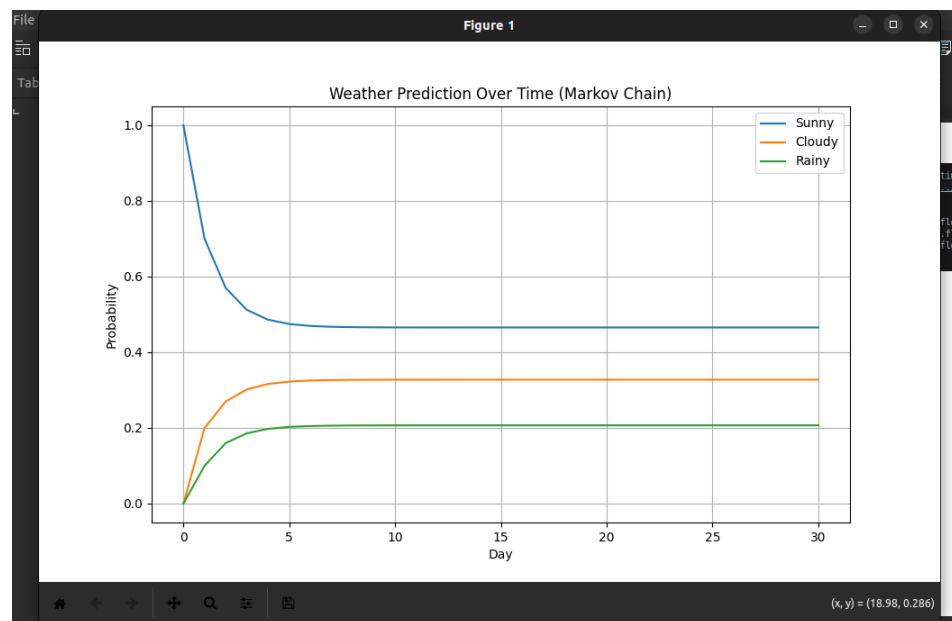
# Output stationary distribution
print("Stationary Distribution (Long-Term Weather Probabilities):")
for i in range(len(states)):
    print(f"{states[i]}: {stationary_dist[i]:.4f}")

```

```

KeyboardInterrupt
complab3-17@complab3-17:~/Documents/time$ cd /home/complab3-17/Documents/time ; /usr/bin/env python3 -m ms-python-debugpy-2025.10.0-linux-x64/bundled/libs/debugpy/adapters/python/debugpy_launcher/main.py
Transition Matrix:
Sunny -> {'Sunny': np.float64(0.7), 'Cloudy': np.float64(0.2), 'Rainy': np.float64(0.1)}
Cloudy -> {'Sunny': np.float64(0.3), 'Cloudy': np.float64(0.4), 'Rainy': np.float64(0.3)}
Rainy -> {'Sunny': np.float64(0.2), 'Cloudy': np.float64(0.5), 'Rainy': np.float64(0.3)}

```



References	Norris, J. R. (1997). <i>Markov Chains</i> . Cambridge University Press. Kemeny, J. G., & Snell, J. L. (1976). <i>Finite Markov Chains</i> . Springer.